1. What is Container Orchestration?

Professional Answer

Container orchestration is the automated management of containerized applications, including deployment, scaling, networking, load balancing, self-healing, and lifecycle management across a cluster of machines. Tools like Kubernetes, Docker Swarm, and Nomad help manage containers efficiently in production environments.

Unprofessional Answer

It just runs containers automatically and keeps them alive.

Interviewer Expectation

They want **automation + scale + production readiness**, not just "running containers".

2. Why do we need Container Orchestration?

Professional Answer

Container orchestration is required to handle large-scale containerized applications where manual management becomes impractical. It ensures high availability, auto-scaling, fault tolerance, rolling updates, service discovery, and efficient resource utilization.

Unprofessional Answer

Because there are too many containers and it's hard to manage them.

Tip

Avoid emotional words like *hard*, *too many*. Use **business and operational reasons**.

3. What problems does Container Orchestration solve?

Professional Answer

Container orchestration solves problems such as:

- Manual deployment errors
- Downtime due to container failures
- Inefficient resource utilization
- Complex networking between containers
- Difficulty in scaling applications

Unprofessional Answer

Containers crash and orchestration fixes it.

4. Name popular Container Orchestration tools.

Professional Answer

Popular container orchestration tools include:

- Kubernetes (most widely adopted)
- Docker Swarm
- HashiCorp Nomad
- Amazon EKS
- Azure AKS
- Google GKE

Unprofessional Answer

Kubernetes and some other tools.

5. Why is Kubernetes more popular than Docker Swarm?

Professional Answer

Kubernetes offers advanced features such as auto-scaling, self-healing, declarative configuration, strong community support, extensibility, and cloud-provider neutrality, making it suitable for complex production workloads.

Unprofessional Answer

Because everyone uses Kubernetes now.

6. What do you mean by "Self-Healing" in orchestration?

Professional Answer

Self-healing refers to the orchestration system's ability to automatically detect failed containers or nodes and restart, reschedule, or replace them to maintain the desired application state.

Unprofessional Answer

If a container dies, Kubernetes restarts it.

7. Explain Desired State in Container Orchestration.

Professional Answer

Desired state is a declarative configuration that defines how the system should look (number of replicas, resource limits, image versions). The orchestrator continuously works to match the actual state to the desired state.

Unprofessional Answer

It's what we want the system to do.

8. What is a Cluster in Container Orchestration?

Professional Answer

A cluster is a group of worker nodes managed by a control plane that runs containerized workloads and provides scheduling, networking, and resource management.

Unprofessional Answer

It's a group of servers.

9. What is Scheduling in Container Orchestration?

Professional Answer

Scheduling is the process of assigning containers or pods to appropriate nodes based on resource availability, constraints, affinity rules, and policies.

Unprofessional Answer

Kubernetes decides where containers run.

10. How does orchestration help in scaling applications?

Professional Answer

Orchestration enables horizontal and vertical scaling by automatically adjusting the number of container replicas or resources based on traffic, CPU, memory usage, or custom metrics.

Unprofessional Answer

It increases containers when traffic increases.

11. What is Service Discovery?

Professional Answer

Service discovery allows containers to dynamically find and communicate with each other using stable service names instead of hardcoded IP addresses.

Unprofessional Answer

Containers find each other automatically.

12. What is Load Balancing in Container Orchestration?

Professional Answer

Load balancing distributes incoming traffic across multiple container instances to ensure high availability, fault tolerance, and optimal performance.

Unprofessional Answer

Traffic is shared between containers.

13. Explain Rolling Updates.

Professional Answer

Rolling updates allow applications to be updated gradually by replacing old container versions with new ones without causing downtime.

Unprofessional Answer

Containers are updated one by one.

14. What is Blue-Green Deployment?

Professional Answer

Blue-Green deployment maintains two identical environments where traffic is switched from the old version (blue) to the new version (green), enabling zero-downtime releases and easy rollback.

Unprofessional Answer

Two environments are used for deployment.

15. What is Canary Deployment?

Professional Answer

Canary deployment releases a new version to a small subset of users before rolling it out to the entire system, reducing risk and validating performance.

Unprofessional Answer

We test new version on few users.

16. How does Container Orchestration improve High Availability?

Professional Answer

It improves availability by distributing workloads across nodes, automatically rescheduling failed containers, using health checks, and ensuring multiple replicas of services.

Unprofessional Answer

If one container fails, another runs.

17. What is Resource Management in Orchestration?

Professional Answer

Resource management ensures efficient allocation of CPU and memory using requests, limits, quotas, and scheduling policies to avoid overcommitment and starvation.

Unprofessional Answer

It controls CPU and RAM.

18. What security benefits does orchestration provide?

Professional Answer

Orchestration platforms provide isolation, role-based access control (RBAC), secrets management, network policies, and secure container execution.

Unprofessional Answer

It is more secure than Docker.

19. What happens if a node goes down?

Professional Answer

The orchestrator detects the node failure and reschedules affected containers on healthy nodes to maintain the desired state.

Unprofessional Answer

Containers move to another server.

20. Real-World Scenario Question

Q: Your application traffic suddenly spikes 5x. How does container orchestration help?

Professional Answer

The orchestration platform triggers auto-scaling policies, increases container replicas, balances traffic across instances, and ensures system stability without manual intervention.

Unprofessional Answer

More containers will start automatically.

21. What is Kubernetes?

Professional Answer

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, networking, self-healing, and lifecycle management of containerized applications across a distributed cluster.

Unprofessional Answer

Kubernetes is used to run Docker containers.

22. Why is Kubernetes needed?

Professional Answer

Kubernetes is needed to manage containerized applications at scale by solving challenges like manual deployments, downtime, scaling, service discovery, fault tolerance, and infrastructure dependency.

Unprofessional Answer

Because managing containers manually is difficult.

23. What kind of problems does Kubernetes solve?

Professional Answer

Kubernetes solves:

- Container failures
- Scaling issues
- Manual deployments
- Service discovery challenges
- Resource underutilization
- High availability requirements

Unprofessional Answer

Containers crash and Kubernetes fixes it.

24. What are the key features of Kubernetes?

Professional Answer

- Automated deployment and rollback
- Auto-scaling
- Self-healing
- Service discovery & load balancing
- Configuration and secret management
- Declarative infrastructure

Unprofessional Answer

Scaling, deployment, and restart.

25. What is declarative configuration in Kubernetes?

Professional Answer

Declarative configuration allows users to define the desired state of the system in YAML manifests, and Kubernetes continuously reconciles the actual state to match it.

Unprofessional Answer

We write YAML files.

Explain Kubernetes architecture.

Professional Answer

Kubernetes follows a master-worker architecture consisting of a Control Plane that manages the cluster and Worker Nodes that run application workloads.

27. What are the main components of Kubernetes Control Plane?

Professional Answer

- API Server
- etcd
- Scheduler
- Controller Manager
- Cloud Controller Manager (optional)

Unprofessional Answer

API server, etcd, scheduler.

28. What are the components of Worker Node?

Professional Answer

- kubelet
- kube-proxy
- Container Runtime (containerd, CRI-O)

Unprofessional Answer

Docker and some services.

---

22. Why did Kubernetes move away from Docker?

Professional Answer

Kubernetes removed Docker as a runtime but continues to support OCI-compliant runtimes like containerd via CRI.

Unprofessional Answer

Docker is not supported anymore.

23. What happens when you create a Pod?

Professional Answer

1. Request goes to API Server
2. API Server validates and stores state in etcd
3. Scheduler assigns node
4. kubelet creates containers
5. kube-proxy sets networking

Explain K8s Architecture in brief.

- Kubernetes follows a control plane and worker node architecture to manage containerized applications efficiently.
- The control plane is responsible for managing the overall cluster state and making global decisions.
- The kube-apiserver acts as the central entry point for all Kubernetes API requests.
- All cluster configuration and state data are stored in etcd, a distributed key-value store.
- etcd serves as the single source of truth for the Kubernetes cluster.
- The **kube-scheduler** monitors the API server for newly created pods that are not assigned to any node.

- The scheduler selects an appropriate worker node based on resource availability and scheduling policies.

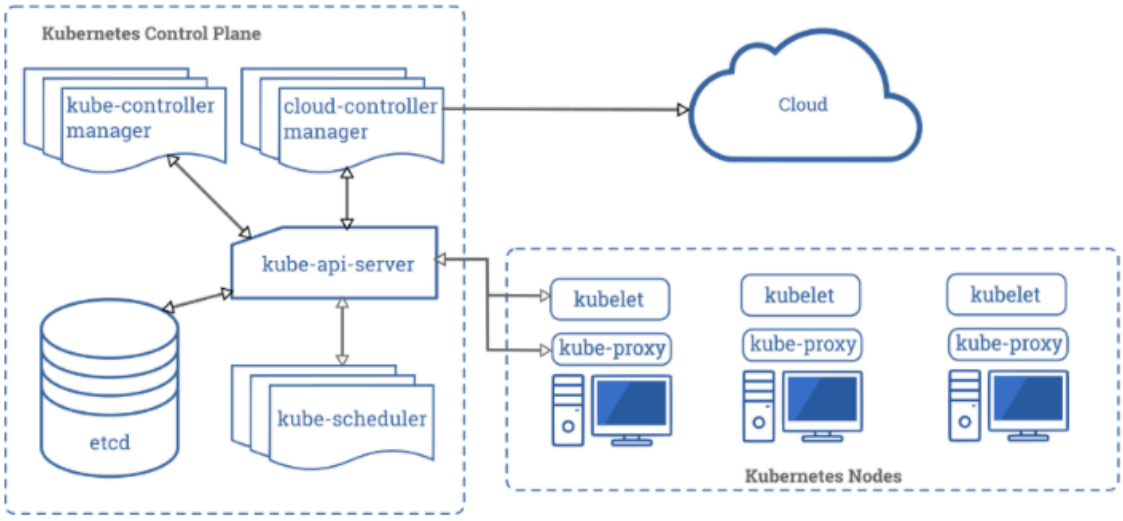
- The **kube-controller-manager** runs multiple controllers that continuously monitor the cluster state.
- Controllers work to ensure the actual state of the cluster matches the desired state defined by users.

- The **cloud-controller-manager** enables integration between Kubernetes and cloud provider services.

- **Worker nodes** are responsible for running application workloads in the form of pods.
- Each worker node runs kubelet, which communicates with the API server.

- kubelet ensures containers are created and running according to pod specifications.

- **The container runtime** pulls container images and starts containers on worker nodes.
- **kube-proxy** manages network rules to enable communication between services and pods.
- Kubernetes assigns a unique IP address to each pod for direct network communication.
- Pods can communicate across nodes without requiring network address translation.
- Node health is continuously monitored through status updates sent to the control plane.
- Failed pods or nodes are automatically recovered through Kubernetes self-healing mechanisms.
- This architecture provides scalability, high availability, and fault tolerance for containerized applications.



LINE 1: kubectl → kube-apiserver

Explanation :

kubectl communicates with the Kubernetes cluster by sending **REST API requests** to the kube-apiserver, which acts as the single entry point for all cluster operations.

Keywords to include:

✓ REST API

✓ Single entry point

✓ Authentication & authorization

LINE 2: kube-apiserver → Authentication & Authorization

Explanation:

The API Server authenticates the user and authorizes the request using RBAC, ensuring only permitted actions are performed on cluster resources.

LINE 3: kube-apiserver → etcd

Explanation:
After validation, the API Server stores the desired state of the cluster in etcd, which acts as the persistent and consistent data store for Kubernetes.

Exam Tip:
Always mention “source of truth” for etcd.

LINE 4: kube-scheduler → kube-apiserver

Explanation:
The scheduler continuously watches the API Server for newly created pods without assigned nodes and selects the most suitable node based on scheduling policies.

LINE 5: kube-scheduler → Worker Node Selection

Explanation:
The scheduler evaluates node resources, taints, tolerations, affinity rules, and constraints before assigning the pod to a worker node.

LINE 6: kube-apiserver → kubelet

Explanation:
Once a node is assigned, the API Server informs the kubelet running on the selected worker node about the pod specification.

LINE 7: kubelet → Container Runtime

Explanation:
kubelet instructs the container runtime (such as containerd) to pull container images, create containers, and start them according to the pod definition.

LINE 8: kubelet → Pod Health Monitoring

Explanation:
kubelet continuously monitors container health using liveness and readiness probes and restarts containers if failures are detected.

LINE 9: kube-proxy → Networking Rules

Explanation:
kube-proxy configures network rules using iptables or IPVS to enable service discovery and load balancing between pods.

LINE 10: Pod ↔ Pod Communication

Explanation:
Each pod is assigned a unique IP address, allowing direct pod-to-pod communication across nodes without network address translation.

LINE 11: Controller Manager → API Server

Explanation:
The controller manager runs control loops that monitor cluster state via the API Server and take corrective actions to match the desired state.

LINE 12: ReplicaSet Controller → Pods

Explanation:
If the number of running pods differs from the desired replica count, the ReplicaSet controller creates or deletes pods to maintain consistency.

LINE 13: Node Controller → Worker Nodes

Explanation:
The Node Controller detects node failures and marks nodes as NotReady if heartbeats from kubelet are not received.

LINE 14: cloud-controller-manager → Cloud Provider

Explanation:
The cloud controller manager interacts with cloud provider APIs to manage load balancers, persistent disks, and node lifecycle operations.

LINE 15: Status Updates → kube-apiserver

Explanation:
kubelet and controllers continuously send status updates to the API Server, ensuring real-time cluster visibility.

Kubernetes follows a control plane and worker node architecture where kube-apiserver acts as the central management component. User requests are validated and stored in etcd, which maintains the cluster state. The scheduler assigns pods to suitable nodes, while kubelet ensures containers are running as defined. Controllers continuously reconcile actual state with desired state, and kube-proxy manages networking and service discovery. This architecture enables scalability, fault tolerance, and self-healing.

What is **CNI (Networking Plugin Internals)**

What CNI does

- Assigns IP addresses to pods
- Sets up routing
- Manages network policies

Internal Components

- CNI binary
- CNI config file
- Network backend

Popular CNIs

- Calico (policy + routing)
- Cilium (eBPF)
- Flannel (simple overlay)

What is CSI (Storage Internals)

CSI Purpose

- Decouples storage from Kubernetes core

Internal Flow

PVC → StorageClass → CSI Driver → Storage Backend

CSI Components

- Controller plugin
- Node plugin

End-to-End Internal Flow

1. kubectl → API server
2. API server → auth + admission
3. State stored in etcd
4. Scheduler assigns node
5. kubelet creates pod
6. Runtime starts containers
7. kube-proxy updates networking
8. Controllers reconcile state

Kubernetes is a distributed, event-driven system where the API server coordinates declarative state stored in etcd, controllers reconcile that state, the scheduler decides placement, and kubelet executes workloads on worker nodes.

Is Kubernetes only for microservices?

Professional Answer

No, Kubernetes can run both microservices and monolithic applications, although it is better suited for distributed workloads.

Unprofessional Answer

Yes, Kubernetes is for microservices.

Is Kubernetes replacing DevOps?

Professional Answer

No, Kubernetes is a platform that enables DevOps practices but does not replace DevOps culture or processes.

Unprofessional Answer

Yes, Kubernetes is DevOps.